

## TEMA 2. Encapsulación

---

1. En Programación Orientada a Objetos (POO), ¿Qué buscan la **encapsulación** y la **ocultación** de información? Enumera brevemente algunas ventajas de la ocultación de información.

---

### 1. Encapsulación y ocultación de información

La **encapsulación** busca agrupar en una única unidad (la clase) tanto los datos como las operaciones que manipulan dichos datos. A diferencia de las estructuras en C, donde los datos y las funciones suelen estar dispersos, en la programación orientada a objetos se persigue que el objeto sea una entidad autónoma. Esto permite tratar al objeto como una unidad que sabe gestionar su propia información y comportamiento de manera cohesionada.

Por su parte, la **ocultación de información** tiene como objetivo proteger el estado interno del objeto frente a accesos indebidos o accidentales desde el exterior. Se busca restringir la visibilidad de los atributos para que solo puedan ser consultados o modificados a través de mecanismos controlados. Esto evita que el código externo dependa de detalles de implementación que podrían variar, garantizando que el uso del objeto sea independiente de cómo esté construido internamente.

Las ventajas de la ocultación de información incluyen:

- **Integridad de datos:** Se asegura que los atributos solo tomen valores válidos mediante filtros en los métodos de acceso, evitando estados inconsistentes que en C ocurrirían al manipular un `struct` directamente.
  - **Mantenibilidad:** Es posible cambiar la estructura interna de la clase (nombres de variables o tipos de datos) sin afectar al resto del programa, siempre que se mantenga la forma de interactuar con ella.
  - **Reducción de la complejidad:** El usuario de la clase solo necesita conocer qué hace el objeto y no cómo lo hace, lo que facilita la comprensión del sistema global al ocultar detalles técnicos innecesarios.
- 

### 2. La interfaz pública y su relación con la ocultación

La **interfaz pública** de un objeto o clase se define como el conjunto de métodos y constantes que son accesibles desde el exterior. Interfaz pública --> Todo lo que no es absolutamente privado. Representa el "contrato" o punto de contacto que la clase ofrece a otros componentes para interactuar con ella. En Java, esta interfaz se compone principalmente de los métodos marcados con el modificador `public`, los cuales dictan qué acciones puede realizar el objeto sin revelar su funcionamiento interno.

La relación con la ocultación de información es de complementariedad: mientras la ocultación mantiene los datos sensibles y la lógica compleja en privado (`private`), la interfaz pública expone solo lo estrictamente necesario para que el objeto sea funcional. Se puede visualizar como la fachada de un módulo que oculta los engranajes internos, permitiendo una comunicación segura y simplificada entre las distintas partes del software.

---

### 3. Diseño de la interfaz pública y facilidad de cambio

Se debe ser consciente y diseñar con extremo cuidado la interfaz pública porque constituye un compromiso permanente con el resto del sistema. Una vez que otros módulos comienzan a utilizar los métodos públicos de una clase, cualquier modificación en sus firmas (nombre, parámetros o tipo de retorno) provocará errores de compilación en todas las partes del código que dependan de ellos. Es el punto de mayor acoplamiento en el programa.

Por lo tanto, **no es fácil cambiar la interfaz pública** una vez que ha sido desplegada o utilizada por terceros. Mientras que el código privado (los detalles ocultos) puede refactorizarse libremente sin consecuencias externas, la interfaz pública debe permanecer estable para garantizar la compatibilidad. Un diseño deficiente que exponga demasiados detalles obligará a realizar cambios costosos y complejos en el futuro ante cualquier evolución del sistema.

## 2. ¿Qué se entiende por la **interfaz pública** de un objeto o clase en POO? Describe brevemente cómo se relaciona con la ocultación de información.

La **interfaz pública** de una clase se define como el conjunto de métodos y constantes que son accesibles desde cualquier otra parte del programa. Representa el "contrato" o el catálogo de servicios que el objeto ofrece a los demás componentes para interactuar con él. En Java, esta interfaz se materializa principalmente a través de los miembros declarados con el modificador de acceso **public**.

La relación con la ocultación de información es de complementariedad directa. Mientras que la ocultación mantiene los detalles de implementación y los datos sensibles en privado (usando **private**), la interfaz pública es el único canal autorizado para interactuar con esos datos. Es el mecanismo que permite que un objeto sea utilizado como una "caja negra", donde se conoce qué se puede hacer con él, pero no necesariamente cómo se ejecutan esas acciones internamente.

Esta separación permite que la interfaz pública actúe como un filtro de seguridad. Al no permitir el acceso directo a los atributos (estado interno), la interfaz pública obliga a que cualquier cambio de estado pase por métodos que pueden contener lógica de validación. Esto asegura que, aunque los detalles internos cambien para mejorar el rendimiento o corregir errores, la forma en que el resto del sistema interactúa con el objeto permanezca constante.

En términos prácticos, una interfaz pública bien diseñada es minimalista: expone solo lo estrictamente necesario. Esto reduce el acoplamiento entre clases, ya que el código externo depende de la interfaz (el "qué") y no de la implementación (el "cómo"). Si se respeta esta separación, es posible rediseñar por completo el interior de una clase sin que se produzca un impacto negativo en las demás partes del software.

## 3. Brevemente: ¿Por qué hay que ser conscientes y diseñar con cuidado la **interfaz pública** de una clase? ¿Es fácil cambiarla?

La **interfaz pública** de una clase, compuesta por sus métodos **public**, constituye el "contrato" que dicha clase firma con el resto del programa. Al diseñar esta interfaz, se decide qué servicios se ofrecen y cómo deben invocarse. Un diseño descuidado puede exponer detalles de la implementación interna que deberían haber permanecido ocultos, lo que obliga a los usuarios de la clase a depender de cómo está construida por dentro, en lugar de qué es lo que hace.

La dificultad de cambiar una interfaz pública radica en el impacto que tiene sobre el código externo. En C, si se cambia la firma de una función en un archivo de cabecera (**.h**), todos los módulos que la llaman dejarán de

compilar. En Java sucede lo mismo: una vez que otros programadores (o incluso otras partes de nuestro propio proyecto) empiezan a usar un método público, ese método se vuelve extremadamente difícil de modificar o eliminar sin romper la compatibilidad de todo el sistema.

Por el contrario, la implementación interna (el código privado) es fácil de cambiar siempre que la interfaz pública se mantenga estable. Si se decide optimizar un algoritmo o cambiar una estructura de datos interna, esto se puede hacer de forma transparente para el resto del programa. Por ello, una interfaz pública debe ser mínima, clara y bien pensada, actuando como una frontera rígida que permite que el interior de la clase sea flexible y evolutivo.

#### 4. ¿Qué son las **invariantes de clase** y por qué la ocultación de información nos ayuda?

Las **invariantes de clase** son condiciones o reglas lógicas que deben cumplirse durante toda la vida del objeto para que se considere en un estado válido. Por ejemplo: Persona debe tener edad mayor o igual que cero. Se pueden comparar con las restricciones de integridad en una base de datos o las reglas de validación en C; por ejemplo, en una clase que represente una **Fecha**, una invariante sería que el valor del mes siempre esté en el rango de 1 a 12. Estas reglas se establecen durante la construcción del objeto y deben preservarse tras la ejecución de cualquier método público.

La ocultación de información es la herramienta fundamental para proteger estas invariantes. Si los atributos fueran públicos (como ocurre por defecto en las **struct** de C), cualquier parte del código podría asignar valores que rompan la lógica del objeto, como establecer un radio negativo en una clase **Circulo**. Al declarar los atributos como **private**, se garantiza que la única forma de alterar el estado sea a través de métodos controlados que validen los datos antes de realizar cualquier cambio.

Esta protección actúa como un "filtro de seguridad". El objeto deja de ser una simple estructura pasiva de datos para convertirse en una entidad inteligente capaz de rechazar instrucciones que lo conducirían a un estado incoherente. En programas complejos, esto reduce drásticamente la aparición de errores en tiempo de ejecución, ya que no es necesario buscar en miles de líneas de código quién cambió un valor de forma incorrecta; la responsabilidad recae únicamente en la lógica interna de la clase.

Además, el uso de la ocultación de información para mantener invariantes facilita la depuración y el mantenimiento. Si una regla de negocio cambia —por ejemplo, si el saldo mínimo de una cuenta pasa de 0 a 100—, solo es necesario modificar el método **setter** o el constructor de la clase correspondiente. Gracias a que el acceso directo estaba restringido, existe la certeza de que ningún otro punto del programa está saltándose esa regla, manteniendo la estabilidad de todo el sistema.

#### 5. Pon un ejemplo de una clase **Punto** en **Java**, con dos coordenadas, **x** e **y**, de tipo **double**, con un método **calcularDistanciaAOrigen**, y que haga uso de la ocultación de información. ¿Cuál es la interfaz pública de la clase **Punto**? ¿Qué significa **public** y **private**?

Para implementar la clase **Punto** siguiendo los principios de la encapsulación, se deben ocultar los detalles de almacenamiento (coordenadas) y exponer únicamente lo necesario. En Java, esto se logra declarando los atributos como privados y los métodos de acceso o cálculo como públicos.

```

public class Punto {
    // Atributos privados: ocultación de información
    private double x;
    private double y;

    // Constructor
    public Punto(double x, double y) {
        this.x = x;
        this.y = y;
    }

    // Interfaz pública: Métodos para interactuar con el objeto
    public double getX() { return x; }
    public double getY() { return y; }

    public double calcularDistanciaAOrigem() {
        // Uso de LaTeX para la fórmula matemática:
        //  $\sqrt{x^2 + y^2}$ 
        return Math.sqrt(Math.pow(x, 2) + Math.pow(y, 2));
    }
}

```

La **interfaz pública** de esta clase está constituida por el constructor `Punto(double, double)`, los métodos *getter* `getX()` y `getY()`, y el método de servicio `calcularDistanciaAOrigem()`. Es, esencialmente, el conjunto de "botones" y "pantallas" que un programador externo puede ver y utilizar. Cualquier cambio interno en cómo se guardan los datos (por ejemplo, si se decidiera usar coordenadas polares internamente) no afectaría a quien usa estos métodos, siempre que la firma de la interfaz se mantenga igual.

El modificador `private` actúa como un muro de privacidad: indica que el miembro (variable o método) solo es accesible desde dentro de la propia clase. Es comparable a una variable local dentro de un archivo `.c` que no ha sido exportada en el `.h`. Por otro lado, `public` es la declaración de apertura al exterior: cualquier otra clase del programa puede invocar esos métodos. En diseño orientado a objetos, lo ideal es mantener el `private` para los datos y el `public` para el comportamiento, forzando así que toda interacción sea controlada y segura.

Esta separación permite que el objeto mantenga su autonomía. Si en el futuro se necesitara añadir una validación (por ejemplo, que las coordenadas no puedan ser infinitas), se podría hacer dentro del constructor o de los métodos de asignación sin que el resto del código que ya usa la clase `Punto` se entere del cambio lógico.

## 6. En Java, ¿A quiénes se pueden aplicar los modificadores `public` o `private`?

Public --> Clases, atributos y métodos. Private --> Clases internas (no las estamos viendo), atributos y métodos.

Los modificadores de acceso en Java tienen un alcance versátil, pero su aplicación varía según el nivel del código en el que se encuentren. Principalmente, se aplican a los **miembros de la clase** (atributos y métodos)

y a las **clases mismas**, aunque con ciertas restricciones importantes que aseguran la coherencia del sistema.

A nivel de **miembros**, se pueden marcar variables, métodos e incluso constructores como **public** o **private**. Un atributo **private** es el estándar para la encapsulación, mientras que un método **private** suele ser una función auxiliar que ayuda a otros métodos internos pero que no debe ser vista desde fuera. Por el contrario, los elementos **public** definen la funcionalidad que la clase "promete" realizar para otros objetos.

A nivel de **clases de nivel superior**, la aplicación es más limitada: una clase solo puede ser **public** o tener acceso por defecto (*package-private*). No es posible declarar una clase principal como **private**, ya que esto impediría que el sistema pudiera instanciarla o usarla de cualquier forma, invalidando su existencia. Sin embargo, en conceptos más avanzados como las **clases anidadas** (una clase dentro de otra), sí se permite el uso de **private** para que esa subclase sea una herramienta exclusiva de la clase que la contiene.

Es fundamental distinguir que estos modificadores **no se aplican a las variables locales** (las que se declaran dentro de un método). En C, una variable dentro de una función tiene su propio ámbito; en Java ocurre igual, y añadirle **public** o **private** resultaría en un error de compilación, ya que su visibilidad está intrínsecamente limitada a la ejecución de ese bloque de código.

## 7. En POO, la visibilidad puede ser pública o privada, pero ¿existen más tipos de visibilidad? ¿Qué ocurre en Java? ¿Y en otros lenguajes?

Aunque la dicotomía entre **public** y **private** es la base de la encapsulación, la mayoría de los lenguajes orientados a objetos ofrecen niveles intermedios para gestionar la confianza entre clases. En Java, además de estos dos, existen **protected** y el acceso por defecto (*\*\*package-private* o *friendly*). El acceso por defecto se aplica cuando no se escribe ningún modificador y permite que cualquier clase dentro del mismo "paquete" (carpeta) acceda a los miembros, facilitando la colaboración entre clases estrechamente relacionadas sin abrirse a todo el programa.

El modificador **protected** es un paso más allá en la jerarquía: permite el acceso a las clases del mismo paquete y, además, a cualquier clase que herede de ella (subclases), incluso si están en paquetes diferentes. Esto es vital para la extensibilidad, permitiendo que un "hijo" vea las herramientas de su "padre" pero manteniendo esas herramientas ocultas para el resto del mundo. En términos de C, sería como tener un archivo de cabecera que solo es incluido por archivos muy específicos que comparten una jerarquía lógica.

En otros lenguajes, el abanico de visibilidad varía según la filosofía de diseño. Por ejemplo, en **C++** (la evolución natural de lo que ya conoces), existen **public**, **private** y **protected**, pero su comportamiento depende de cómo se herede la clase. En lenguajes como **C#**, existe además el modificador **internal**, que limita la visibilidad a un mismo "ensamblado" (un archivo **.exe** o **.dll**), lo cual es muy útil para proteger bibliotecas de software.

Incluso existen lenguajes con filosofías radicales como **Python**, donde la privacidad estricta no existe a nivel de compilador. En Python, se usa una convención de nombres (un guion bajo antes de la variable, como **\_secreto**) para "pedir amablemente" a otros programadores que no toquen ese dato. Esto demuestra que, aunque los mecanismos técnicos cambien, el concepto de **encapsulación** como principio de diseño es universal en la informática moderna.

## 8. Responde: Los miembros de instancia privados de un objeto están ocultos para (a) otras clases o (b) otras instancias, aunque sean de la

misma clase. Pon un ejemplo añadiendo un método `calcularDistanciaAPunto(Punto otro)` y explica la respuesta.

La respuesta correcta es la **(a): otras clases**. En Java, la visibilidad de los miembros privados se define a nivel de **clase**, no de instancia (objeto). Esto significa que un objeto de la clase `Punto` tiene "permiso" para acceder a los atributos `private` de cualquier otro objeto de la misma clase `Punto`.

Este comportamiento es muy útil y lógico desde el punto de vista del diseño. Dado que el código del método está escrito dentro de la propia clase, el programador que diseñó `Punto` conoce perfectamente cómo funcionan sus atributos privados. No hay riesgo de romper la integridad del objeto ajeno porque se está operando bajo las mismas reglas y lógica de validación que se definieron para la propia clase.

A continuación se presenta el ejemplo solicitado, integrando el método que interactúa con otra instancia:

```
public class Punto {
    private double x;
    private double y;

    public Punto(double x, double y) {
        this.x = x;
        this.y = y;
    }

    public double calcularDistanciaAPunto(Punto otro) {
        // Se accede directamente a otro.x y otro.y aunque sean private.
        // Esto es posible porque estamos dentro de la clase Punto.
        double dx = this.x - otro.x;
        double dy = this.y - otro.y;

        return Math.sqrt(Math.pow(dx, 2) + Math.pow(dy, 2));
    }
}
```

Como se observa en el código, el objeto actual (`this`) puede "leer" los datos internos del objeto `otro` sin necesidad de usar métodos *getters*. Si se intentara hacer esto mismo desde una clase llamada `Mapa` o `Calculadora`, el compilador daría un error inmediatamente. Por lo tanto, la privacidad en Java protege la estructura interna frente a extraños, pero permite la colaboración directa entre semejantes.

## 9. ¿Qué son los métodos "getter" y "setter" en los lenguajes orientados a objetos?

Los métodos **getter** y **setter** son funciones públicas que actúan como intermediarios entre los atributos privados de una clase y el mundo exterior. En lugar de permitir que un programador acceda directamente a una variable (como harías con un `punto.x` en una `struct` de C), se le obliga a pasar por estos métodos. El *getter* tiene la misión de recuperar y devolver el valor de un atributo, mientras que el *setter* se encarga de recibir un nuevo valor y asignarlo a la variable interna.

El uso de estos métodos no es un capricho estético, sino que responde al principio de **encapsulación**. Al centralizar el acceso, se gana el control total sobre qué ocurre cuando se consulta o modifica un dato. Por ejemplo, un *getter* podría devolver una copia de un objeto para evitar que el original sea modificado externamente, o un *setter* podría registrar un log cada vez que una variable crítica cambia de valor, algo que sería imposible de rastrear si el acceso fuera directo.

Una de las ventajas más potentes es la **validación y transformación**. Dentro de un *setter*, se puede incluir lógica de control para asegurar que el objeto siempre esté en un estado válido. Si se intenta asignar una edad negativa o un nombre vacío, el *setter* puede rechazar la operación. Además, permiten crear atributos de "solo lectura" (implementando solo el *getter*) o de "solo escritura" (solo el *setter*), ofreciendo una flexibilidad que las estructuras simples de datos no poseen.

En Java, existe una convención de nombres muy estricta para estos métodos (conocida como el estándar *JavaBeans*). Si un atributo se llama **velocidad**, el *getter* debe llamarse `getVelocidad()` y el *setter* `setVelocidad(tipo valor)`. Seguir esta norma es vital, ya que muchas herramientas y librerías de Java utilizan la reflexión para buscar estos nombres automáticamente y poder manipular los objetos de forma dinámica.

## 10. Cuando nos referimos a que la ocultación de información mejora la "seguridad" del programa, ¿nos referimos a que no pueda ser "hackeado"?

No, en el contexto de la ingeniería de software y la POO, el término **"seguridad"** no se refiere principalmente a la ciberseguridad o a la protección contra ataques malintencionados (hackeos), sino a la **robustez y fiabilidad** del código. Se trata de una seguridad "estructural" que impide que el propio programador, o sus compañeros de equipo, cometan errores accidentales que dejen al sistema en un estado inconsistente o propenso a fallos.

En lenguajes como C, la falta de ocultación de información puede compararse con un coche donde cualquier pasajero puede tocar los cables del motor mientras el vehículo está en marcha. Si un programador modifica una variable crítica desde fuera sin conocer las dependencias internas, puede provocar un "efecto dominó" de errores difíciles de rastrear. La ocultación de información pone una "carcasa" al motor, dejando a la vista solo el volante y los pedales (la interfaz pública), lo que garantiza que el sistema se use siempre de la forma prevista.

Dicho esto, la encapsulación sí aporta una capa de seguridad indirecta frente a vulnerabilidades. Al centralizar el acceso a los datos en métodos específicos (como los *setters*), es mucho más sencillo implementar filtros de saneamiento de datos. Por ejemplo, si una clase gestiona una base de datos, el método que recibe el texto puede validar que no contenga caracteres sospechosos, actuando como un primer muro de contención antes de que el dato llegue a las capas internas del programa.

En resumen, cuando hablamos de seguridad en encapsulación, nos referimos a minimizar la **fragilidad del software**. Un programa seguro es aquel donde los cambios en una parte no rompen accidentalmente otras diez, y donde los objetos son "inteligentes" y responsables de mantener su propia integridad. Esta disciplina reduce los *bugs* de lógica, que son, irónicamente, los puntos de entrada más comunes para los fallos reales de seguridad.

## 11. ¿Qué diferencia hay entre **miembro de instancia** y **miembro de clase**? ¿Los miembros de clase también se pueden ocultar?



La distinción fundamental reside en **quién es el propietario** del dato o del comportamiento. Un **miembro de instancia** pertenece a cada objeto individual creado; por ejemplo, en una clase **Coche**, el número de bastidor es de instancia porque cada coche tiene el suyo propio. En cambio, un **miembro de clase** (declarado con la palabra reservada **static**) pertenece a la estructura de la clase en sí, no a un objeto concreto. Es una única variable o método compartido por todos los objetos de ese tipo.

Para un programador de C, esta diferencia es muy intuitiva si se compara con el ámbito de las variables. Los miembros de instancia son equivalentes a los campos dentro de una **struct**, que se replican cada vez que declaras una variable de ese tipo. Los miembros de clase (**static**) funcionan de manera similar a una **variable global** o una variable **static** dentro de un archivo **.c**: solo existe una copia en toda la memoria del programa, independientemente de cuántas estructuras crees.

Respecto a la segunda cuestión, los miembros de clase **también se pueden (y deben) ocultar**. El hecho de que un atributo sea **static** no significa que deba ser público. Por ejemplo, si una clase **Empleado** tiene un contador estático para asignar IDs automáticos, ese contador debe ser **private static**. De lo contrario, cualquier parte del programa podría resetear el contador a cero, rompiendo la lógica de asignación de IDs para los futuros empleados.

La encapsulación se aplica exactamente igual: se declara el miembro estático como **private** y, si es necesario, se crea un **método estático público** (un *getter* estático) para consultarlo. Esto mantiene la coherencia del diseño, asegurando que incluso los datos compartidos a nivel global dentro de la aplicación sigan las reglas de protección y validación que definimos en nuestra clase.

## 12. Brevemente: ¿Tiene sentido que los constructores sean privados?

A primera vista, podría parecer contradictorio que un **constructor** sea privado, ya que su función es permitir la creación de objetos desde el exterior. Sin embargo, en la Programación Orientada a Objetos, un constructor **private** es una herramienta de diseño extremadamente potente. Su sentido principal es **tomar el control total sobre la instanciación**, impidiendo que cualquier clase externa use el operador **new** de forma indiscriminada.

Al hacer el constructor privado, la propia clase se convierte en la única entidad capaz de crear sus propios objetos. Esto se utiliza para implementar patrones de diseño específicos, como el **Singleton**, donde se garantiza que solo exista una única instancia de una clase en todo el programa (por ejemplo, para una conexión a una base de datos o un sistema de log). En este caso, la clase ofrece un método público y estático que devuelve esa única instancia controlada.

Otro escenario común es la creación de **clases de utilidad**, que solo contienen métodos estáticos (como la clase **Math** de Java). Dado que no tiene sentido crear un objeto de una clase que solo ofrece funciones matemáticas globales, se añade un constructor privado para evitar que alguien malgaste memoria instanciándola por error.

Finalmente, también se utiliza en el patrón de **Métodos de Factoría**. En lugar de usar **new**, se obliga al usuario a llamar a métodos como **crearUsuarioAdministrador()** o **crearUsuarioInvitado()**. Esto permite que la clase realice lógicas complejas de configuración antes de entregar el objeto, asegurando que el "producto" final esté perfectamente construido y validado desde el primer segundo.

## 13. ¿Cómo se indican los **miembros de clase** en Java? Pon un ejemplo, en la clase **Punto** definida anteriormente, para que incluya miembros de



clase que permitan saber cuáles son los valores **x** e **y** máximos que se han establecido en todos los puntos que se hayan creado hasta el momento.

En Java, los **miembros de clase** se indican utilizando la palabra reservada **static**. Al añadir este modificador a un atributo o método, este deja de pertenecer a una instancia específica (un objeto concreto) y pasa a ser propiedad de la clase en su conjunto. En términos de memoria, esto significa que solo existe una copia de ese dato para todos los objetos de esa clase, lo que permite compartir información de manera global entre ellos.

Para implementar el seguimiento de los valores máximos en la clase **Punto**, se deben declarar variables estáticas. Estas variables se actualizarán cada vez que se cree un nuevo punto o se modifiquen sus coordenadas, comparando los valores actuales con los máximos registrados hasta la fecha. Es fundamental mantener estos miembros como **private** para evitar que se alteren externamente de forma incorrecta, cumpliendo con la ocultación de información.

A continuación, se muestra cómo quedaría la clase **Punto** con esta funcionalidad:

```
public class Punto {
    // Miembros de instancia
    private double x;
    private double y;

    // Miembros de clase (compartidos por todos los puntos)
    private static double xMaximo = Double.NEGATIVE_INFINITY;
    private static double yMaximo = Double.NEGATIVE_INFINITY;

    public Punto(double x, double y) {
        this.x = x;
        this.y = y;
        actualizarMaximos(x, y);
    }

    private static void actualizarMaximos(double nuevoX, double nuevoY) {
        if (nuevoX > xMaximo) xMaximo = nuevoX;
        if (nuevoY > yMaximo) yMaximo = nuevoY;
    }

    // Getters estáticos para acceder a los máximos desde fuera
    public static double getXMaximo() { return xMaximo; }
    public static double getYMaximo() { return yMaximo; }
}
```

La lógica de actualización se ha encapsulado en un método privado y estático llamado **actualizarMaximos**. Este método es invocado por el constructor cada vez que nace un nuevo objeto. Gracias a esto, cualquier programador puede consultar **Punto.getXMaximo()** en cualquier momento para obtener el valor más alto registrado, sin necesidad de conocer cuántos objetos existen o cómo se lleva el cálculo internamente.

14. Como sería un método factoría dentro de la clase **Punto** para construir un **Punto** a partir de dos coordenadas, pero que las redondee al entero más cercano. Escribe sólo el código del método, no toda la clase ¿Has usado **static**?

Para implementar un método factoría que construya un **Punto** con coordenadas redondeadas, es **imprescindible** utilizar la palabra reservada **static**. Esto se debe a que el método debe poder invocarse sobre la clase misma (por ejemplo, **Punto.crearPuntoRedondeado(...)**) antes de que el objeto que queremos crear exista siquiera en la memoria.

Aquí se presenta el código del método solicitado:

```
public static Punto crearPuntoRedondeado(double x, double y) {  
    // Se realiza el redondeo al entero más cercano  
    double xRedondeado = Math.round(x);  
    double yRedondeado = Math.round(y);  
  
    // Se invoca al constructor interno y se devuelve la nueva instancia  
    return new Punto(xRedondeado, yRedondeado);  
}
```

Al utilizar este enfoque, se centraliza la lógica de "preprocesamiento" de los datos. El programador que utiliza la clase no necesita preocuparse por redondear los valores manualmente en su código; simplemente elige el método factoría adecuado y la clase **Punto** se encarga de entregarle un objeto que cumple con las condiciones esperadas.

15. Cambia la implementación de **Punto**. En vez de dos **double**, emplea un array interno de dos posiciones, intentando no modificar la interfaz pública de la clase.

Para realizar este cambio, se sustituyen los atributos individuales **x** e **y** por un único atributo de tipo array, por ejemplo, **double[] coordenadas = new double[2]**. Lo fundamental de este ejercicio es demostrar que, gracias a la **encapsulación**, el mundo exterior no percibirá ningún cambio en el comportamiento de la clase, ya que la firma de los métodos públicos permanecerá idéntica.

A continuación se presenta la nueva implementación interna de la clase **Punto**:

```
public class Punto {  
    // Nueva representación interna: los datos están ocultos tras un array  
    private double[] coords;  
  
    public Punto(double x, double y) {  
        this.coords = new double[]{x, y};  
    }  
  
    // La interfaz pública se mantiene igual: los métodos devuelven lo mismo  
    public double getX() {
```

```
        return coords[0];
    }

    public double getY() {
        return coords[1];
    }

    public double calcularDistanciaAOrigen() {
        // La lógica interna cambia para leer el array, pero el resultado es el
        mismo
        return Math.sqrt(Math.pow(coords[0], 2) + Math.pow(coords[1], 2));
    }
}
```

Desde la perspectiva de quien utiliza esta clase, no ha habido ninguna ruptura. Si estuviéramos en C y hubiéramos cambiado una `struct` con campos `x` e `y` por un array `c[2]`, todos los archivos que accedieran a `punto.x` dejarían de compilar inmediatamente. En Java, al haber protegido los datos tras métodos *getter*, el cambio es totalmente transparente y seguro.

Esta capacidad de cambiar la "fontanería" interna sin afectar al "usuario" es lo que permite que el software evolucione. Se podría cambiar el array por una estructura más compleja o incluso obtener las coordenadas de una base de datos en tiempo real, y mientras los métodos `getX()` y `getY()` sigan existiendo, el resto del programa seguirá funcionando sin tocar una sola línea de código.

**16. Si un atributo va a tener un método "getter" y "setter" públicos, ¿no es mejor declararlo público? ¿Cuál es la convención más habitual sobre los atributos, que sean públicos o privados? ¿Tiene esto algo que ver con las "invariantes de clase"?**

A primera vista, podría parecer que un atributo con *getter* y *setter* es equivalente a uno público, pero existe una diferencia conceptual y práctica abismal. Si se declara un atributo como `public`, se pierde para siempre la capacidad de controlar qué valores recibe o quién lo consulta. En cambio, el uso de métodos intermediarios permite que, aunque hoy el acceso sea libre, mañana se pueda añadir una validación, un log de auditoría o una transformación sin romper el código de los demás usuarios de la clase.

La convención más habitual y recomendada en Java (y en la mayoría de lenguajes orientados a objetos) es que **todos los atributos sean `private`**. Incluso si en el momento de la creación no parecen necesitar validación, la buena práctica dicta que se protejan por defecto. Es mucho más sencillo añadir un *getter* público a un atributo privado más adelante que intentar convertir un atributo público en privado una vez que el proyecto ha crecido, ya que esto último obligaría a refactorizar cada punto donde se usó la variable directamente.

Esto tiene una relación directa y crítica con las **invariantes de clase**. Como se mencionó anteriormente, una invariante es una regla que el objeto debe garantizar para ser válido (como que el radio de un círculo no sea negativo). Si el atributo es público, la invariante es "frágil" porque cualquier programador puede saltársela por descuido. Al usar un *setter*, el objeto se convierte en el guardián de su propia integridad, rechazando cualquier valor que rompa sus reglas lógicas mediante un simple `if`.

En resumen, los métodos de acceso proporcionan un **nivel de indirección**. Esta capa extra de software es la que permite que el diseño sea flexible y robusto. Declarar atributos públicos es, en la práctica, delegar la responsabilidad de la coherencia de los datos a los demás, mientras que mantenerlos privados con métodos de acceso es asumir la responsabilidad desde la propia clase.

## 17. ¿Qué significa que una clase sea **immutable**? ¿qué es un método modificador? ¿Un método modificador es siempre un "setter"? ¿Tiene ventajas que una clase sea immutable?

Una clase se considera **immutable** cuando su estado (el valor de sus atributos) no puede ser alterado una vez que el objeto ha sido creado. En términos prácticos, esto significa que la instancia nace con unos datos específicos y los mantiene inalterables durante toda su vida útil. Para lograr esto en Java, se suelen declarar los atributos como **private** y **final**, se omiten los métodos "setter" y se asegura que cualquier operación que parezca modificar el objeto en realidad devuelva una **nueva instancia** con los cambios aplicados, dejando la original intacta.

Un **método modificador** (también llamado *mutator*) es cualquier método que cambia el estado interno de un objeto. Aunque el ejemplo más clásico es el "setter", un método modificador no siempre sigue ese patrón. Por ejemplo, en una clase **Lista**, un método **vaciar()** o **eliminarElemento()** son modificadores porque alteran la información interna, pero no son "setters" en el sentido estricto de la palabra. En una clase immutable, por definición, no existen métodos modificadores que afecten a la instancia actual.

Las ventajas de la inmutabilidad son numerosas, especialmente en sistemas complejos. Al ser objetos que nunca cambian, son intrínsecamente **seguros para el uso de hilos** (*thread-safe*), ya que no hay riesgo de que un hilo modifique un dato mientras otro lo está leyendo. Además, simplifican enormemente la depuración: si un objeto immutable tiene un valor incorrecto, el error tuvo que ocurrir obligatoriamente en el momento de su creación, eliminando la necesidad de rastrear en qué parte del programa se alteró su estado posteriormente.

En Java, un ejemplo muy conocido de clase immutable es **String**. Cuando se realiza una operación como **texto.toUpperCase()**, no se modifica el texto original, sino que se crea y devuelve un nuevo objeto **String** con las letras en mayúsculas. Esta filosofía de diseño previene efectos secundarios inesperados y permite que el sistema comparta instancias de forma eficiente, ahorrando memoria y mejorando la estabilidad general del software.

## 18. ¿Es recomendable incluir métodos "setter" siempre y como convención?

No, no es recomendable incluir métodos "setter" por defecto para todos los atributos. Aunque muchas herramientas de generación automática de código facilitan su creación, hacerlo sin criterio va en contra del principio de **encapsulación**. Un objeto debe exponer solo aquello que es estrictamente necesario para su función; si un atributo no necesita ser modificado desde el exterior, añadirle un *setter* es abrir una vulnerabilidad innecesaria en la integridad de la clase.

La decisión de incluir un *setter* debe basarse en la naturaleza del dato. Existen atributos que son **identificadores** (como un DNI o un número de cuenta) que no deberían cambiar nunca tras la creación del objeto; en estos casos, lo correcto es omitir el *setter* para garantizar la consistencia. Al limitar los métodos modificadores, se reduce la superficie de error y se simplifica el razonamiento sobre el estado del programa, algo que en C solía ser una fuente constante de errores por punteros que modificaban memorias inesperadas.

Además, el exceso de *setters* fomenta lo que se conoce como el "modelo de dominio anémico", donde los objetos son simples bolsas de datos sin inteligencia. En lugar de un `setSaldo(getSaldo() + 100)`, es mucho más robusto diseñar un método con significado de negocio como `depositar(100)`. Este último permite encapsular reglas (como no aceptar depósitos negativos) y ocultar la implementación, mientras que el *setter* genérico expone la gestión del estado al usuario de la clase.

En la práctica moderna, se prefiere tender hacia la **inmutabilidad** o hacia interfaces mínimas. Si un objeto puede funcionar correctamente sin permitir cambios en sus atributos una vez construido, es preferible no proporcionar *setters*. Esta disciplina de diseño hace que el código sea más predecible, más fácil de testear y mucho más resistente a los cambios futuros en los requisitos del sistema.

## 19. ¿La clase `String` en Java es mutable o inmutable? ¿Qué ocurre al concatenar dos cadenas? ¿Qué debemos hacer si vamos a hacer una operación que implique concatenar muchas veces para construir paso a paso una cadena muy larga?

La clase `String` en Java es estrictamente **inmutable**. Esto significa que una vez que se crea un objeto de tipo cadena, su contenido no puede ser modificado bajo ninguna circunstancia, no voy a encontrar ningún método que me permita modificarlo. Esta decisión de diseño permite que Java optimice el uso de memoria a través de un "pool" de cadenas, donde varias variables pueden apuntar a la misma instancia de texto de forma segura, ya que existe la garantía de que nadie la alterará.

Cuando se concatenan dos cadenas, por ejemplo mediante el operador `+`, no se modifica ninguna de las cadenas originales. En su lugar, el entorno de ejecución de Java reserva un nuevo espacio de memoria lo suficientemente grande para albergar el resultado combinado, copia ambos textos y devuelve una **nueva instancia** de `String`. Si se realiza esta operación dentro de un bucle, se generan miles de objetos intermedios que quedan huérfanos en la memoria casi instantáneamente, obligando al recolector de basura (*Garbage Collector*) a trabajar intensamente, lo que degrada drásticamente el rendimiento del programa.

Para construir paso a paso una cadena muy larga, la recomendación técnica es utilizar la clase `StringBuilder`. A diferencia de `String`, `StringBuilder` es **mutable** y gestiona un búfer interno (un array de caracteres) que puede crecer dinámicamente. Al usar su método `.append()`, se añaden caracteres directamente al final del espacio ya reservado, evitando la creación constante de nuevos objetos y las costosas copias de memoria que ocurrirían con la concatenación simple.

Una vez que se ha terminado de construir la cadena completa con `StringBuilder`, se utiliza el método `.toString()` para obtener el resultado final en forma de un `String` inmutable. Este enfoque es similar a cómo se gestionaría un búfer de caracteres dinámico en C mediante `malloc` y `realloc`, pero con la seguridad y facilidad que proporcionan los objetos en Java.

## 20. En POO ¿Cómo se comparan objetos de una misma clase? ¿Por su contenido o por su identidad? ¿Qué es el método `equals` en Java? ¿Qué hace por defecto? ¿Cómo se deben comparar dos cadenas en Java?

En Programación Orientada a Objetos, existen dos formas fundamentales de comparar objetos: por **identidad** o por **contenido**. La identidad (comparación por referencia) utiliza el operador `==` y determina si dos variables apuntan exactamente a la misma posición de memoria, es decir, si son el mismo objeto. El contenido (comparación lógica), por otro lado, verifica si los datos internos de dos objetos distintos son equivalentes, aunque residan en lugares diferentes de la memoria.

En Java, el método **equals** es la herramienta estándar para realizar la comparación por contenido. Se hereda de la clase base **Object** y su propósito es permitir que cada clase defina qué significa que dos de sus instancias sean "iguales". Sin embargo, **por defecto**, el método **equals** de la clase **Object** se comporta exactamente igual que el operador **==**: simplemente compara las direcciones de memoria. Por ello, si se desea comparar objetos por sus atributos (como dos objetos **Punto** con las mismas coordenadas), es obligatorio sobrescribir este método en la propia clase para implementar la lógica de comparación deseada.

Para el caso específico de las **cadenas de texto (String)**, nunca se deben comparar utilizando el operador **==**. Debido a que **String** es un objeto, **==** solo diría si son la misma instancia en el pool de constantes. Para saber si dos cadenas contienen el mismo texto, se debe usar siempre el método **.equals()**. Existe un error muy común al venir de C, donde se intentan comparar punteros o usar funciones externas; en Java, la cadena misma ofrece el método para validarse contra otra.

```
String s1 = new String("hola");
String s2 = new String("hola");

if (s1 == s2) { /* FALSO: son objetos distintos en memoria */ } //Comparando por
identidad
if (s1.equals(s2)) { /* VERDADERO: el contenido es el mismo */ } //Comparando por
contenido
```

Es importante mencionar que existe también el método **equalsIgnoreCase()**, muy útil cuando se quiere comparar el contenido omitiendo las diferencias entre mayúsculas y minúsculas. En resumen, mientras que la identidad (**==**) es una comprobación física de la dirección de memoria, el método **equals** es una comprobación lógica que el programador debe adaptar según la naturaleza de su clase.

## 21. ¿Qué son las clases "wrapper" en un lenguaje de programación orientado a objetos? ¿Cómo se hace? ¿Es un proceso automático? ¿Qué ventajas tienen? ¿Todos los lenguajes orientados a objetos tienen tipos primitivos y necesitan wrappers?

Las clases **wrapper** (o envoltorios) son clases especiales que permiten tratar a los tipos de datos primitivos (como **int**, **double** o **boolean**) como si fueran objetos. En Java, cada tipo primitivo tiene su correspondiente clase envoltorio (por ejemplo, **Integer** para **int**, **Double** para **double**, etc.). Esto es necesario porque existen ciertas estructuras en Java, como las colecciones (listas o mapas), que solo pueden almacenar objetos y no valores primitivos puros.

En versiones modernas de Java, este proceso es **automático** gracias a una funcionalidad denominada **Autoboxing y Unboxing**. El *autoboxing* ocurre cuando el compilador convierte automáticamente un valor primitivo en su objeto equivalente (por ejemplo, al pasar un **int** a una lista de **Integer**). El *unboxing* es el proceso inverso, donde el objeto se extrae automáticamente para volver a ser un valor primitivo cuando se necesita realizar una operación aritmética.

La ventaja principal de los wrappers es que permiten que los tipos básicos se integren en el ecosistema de la Programación Orientada a Objetos. Al ser objetos, estas clases incluyen **métodos de utilidad** muy potentes, como convertir una cadena de texto a un número (**Integer.parseInt("123")**) o realizar comparaciones

avanzadas. Además, permiten el uso de valores nulos (`null`), algo que un primitivo puro no puede representar, lo cual es útil en bases de datos para indicar la ausencia de un valor.

No todos los lenguajes orientados a objetos gestionan esta distinción de la misma manera. Lenguajes como **Smalltalk** o **Ruby** son considerados "puramente orientados a objetos" porque no tienen tipos primitivos; todo, incluso un simple número, es un objeto desde el principio. Java, al igual que **C++**, mantiene los primitivos por una cuestión de rendimiento, ya que operar con un `int` es mucho más rápido y consume menos memoria que gestionar un objeto completo, dejando los wrappers como una herramienta de compatibilidad necesaria.

## 22. ¿En POO qué es un **tipo de dato enumerado**? ¿En Java, un tipo de dato enumerado es una clase? ¿Qué ventajas tienen en términos de encapsulación los enumerados en Java?

Un **tipo de dato enumerado** (`enum`) es un tipo especial de dato que permite definir un conjunto fijo de constantes con nombre. Es un tipo con un número determinado de variables posibles. Se utiliza para representar categorías que no cambian, como los días de la semana, los meses del año, los estados de un pedido (PENDIENTE, ENVIADO, ENTREGADO) o los puntos cardinales. Su objetivo es sustituir el uso de números mágicos (como `1` para "Lunes") o cadenas de texto sueltas, que son propensas a errores tipográficos.

En Java, un enumerado **es una clase**. Esta es una de las diferencias más potentes respecto a lenguajes como C. Al ser una clase, un `enum` puede tener atributos, métodos e incluso constructores. La única restricción es que el conjunto de instancias es predefinido y no se pueden crear nuevos objetos del `enum` en tiempo de ejecución mediante `new`. Definición del profe: En Java, un enumerado es una clase, cuyas instancias son finitas, conocidas de antemano, y tienen un nombre cada una (valor del enumerado, el que aparece en mayúsculas).

En términos de **encapsulación**, los enumerados en Java ofrecen ventajas excepcionales:

- **Seguridad de Tipos (Type Safety):** Evitan que se pasen valores inválidos. Si un método espera un `EstadoPedido`, el compilador no permitirá pasar un `int` o un `String` arbitrario. El rango de valores está estrictamente controlado por la propia clase.
- **Comportamiento propio:** Puedes encapsular lógica dentro del propio enumerado. Por ejemplo, un `enum Moneda` puede tener un atributo `valorEnDolares` y un método `convertir(double cantidad)`, ocultando la complejidad de los tipos de cambio dentro de la definición del tipo.
- **Legibilidad y Mantenimiento:** Al centralizar las constantes y su lógica asociada en un solo lugar, el código es más fácil de entender. Si necesitas añadir una nueva categoría, lo haces en el `enum` y todos los lugares donde se usa se benefician de la nueva estructura sin riesgo de inconsistencias.
- **Uso en Sentencias switch:** Java permite usar enumerados de forma nativa en estructuras de control, lo que hace que el código sea mucho más limpio que usar múltiples comparaciones de objetos o cadenas.

23. Crea un tipo enumerado en Java que se llame `Mes`, con doce posibles instancias y que además proporcione métodos para obtener cuántos días tiene ese mes, el ordinal de ese mes en el año (1-12), empleando atributos privados y constructores del tipo enumerado. Añade además cuatro métodos para devolver si ese mes tiene algunos días de invierno, primavera, verano u otoño, indicando con un booleano el hemisferio (norte o sur, parámetro `enHemisferioNorte`). Es decir:



`esDePrimavera(boolean esHemisferioNorte), esDeVerano(boolean esHemisferioNorte), esDeOtoño(boolean esHemisferioNorte), esDeInvierno(boolean esHemisferioNorte)`

Para resolver este ejercicio, aprovecharemos que en Java los enumerados son clases completas. Definiremos atributos privados para almacenar los días y el número de mes, y utilizaremos el constructor del `enum` para inicializar estos valores.

La lógica de las estaciones es interesante: los meses de "transición" (como marzo o septiembre) suelen tener días de dos estaciones distintas. El método devolverá `true` si el mes contiene **al menos un día** de la estación consultada.

```
public enum Mes {  
    ENERO(31, 1), FEBRERO(28, 2), MARZO(31, 3), ABRIL(30, 4),  
    MAYO(31, 5), JUNIO(30, 6), JULIO(31, 7), AGOSTO(31, 8),  
    SEPTIEMBRE(30, 9), OCTUBRE(31, 10), NOVIEMBRE(30, 11), DICIEMBRE(31, 12);  
  
    private final int dias;  
    private final int numero;  
  
    // El constructor de un enum siempre es privado por defecto  
    Mes(int dias, int numero) {  
        this.dias = dias;  
        this.numero = numero;  
    }  
  
    public int getDias() { return dias; }  
    public int getNumero() { return numero; }  
  
    public boolean esDePrimavera(boolean enHemisferioNorte) {  
        // Norte: Marzo a Junio | Sur: Septiembre a Diciembre  
        return enHemisferioNorte ? (numero >= 3 && numero <= 6) : (numero >= 9 &&  
numero <= 12);  
    }  
  
    public boolean esDeVerano(boolean enHemisferioNorte) {  
        // Norte: Junio a Septiembre | Sur: Diciembre a Marzo  
        return enHemisferioNorte ? (numero >= 6 && numero <= 9) : (numero == 12 ||  
numero <= 3);  
    }  
  
    public boolean esDeOtoño(boolean enHemisferioNorte) {  
        // Norte: Septiembre a Diciembre | Sur: Marzo a Junio  
        return enHemisferioNorte ? (numero >= 9 && numero <= 12) : (numero >= 3 &&  
numero <= 6);  
    }  
  
    public boolean esDeInvierno(boolean enHemisferioNorte) {  
        // Norte: Diciembre a Marzo | Sur: Junio a Septiembre  
        return enHemisferioNorte ? (numero == 12 || numero <= 3) : (numero >= 6 &&  
numero <= 9);  
    }  
}
```

```
}  
}
```

### Análisis de la implementación:

1. **Encapsulación:** Los atributos `dias` y `numero` son `private final`. No se pueden cambiar desde fuera, garantizando que un `Mes` sea inmutable y consistente.
2. **Constructor:** Se ejecuta automáticamente para cada una de las 12 instancias. Nota que no usamos `public` en el constructor, ya que los `enum` no permiten la creación externa de nuevas instancias.
3. **Lógica Hemisférica:** Hemos simplificado la lógica agrupando los 4 meses que tocan cada estación (considerando que las estaciones astronómicas empiezan cerca del día 21 del tercer mes de cada trimestre).